Phage SECURITY

SECURITY REVIEW · PREPARED FOR

Perpgame

A leveraged-token basket protocol on HyperEVM

DATE	CONDUCTED BY	COMMIT	REMEDIATION
17.06.2026	Pyro, Lead Security Researcher BengalCatBalu, Lead Security Researcher MarketerKA, Security Researcher	34c47cb	7dcfba6

01 ABOUT

Phage Security

Phage Security is a team of highly skilled smart-contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike.

With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

PREVIOUS WORK
github.com/phage-security

REACH OUT
Telegram · @Pyro3b

WHAT AN AUDIT IS

An audit is a time, resource, and expertise-bound effort in which trained researchers evaluate smart contracts using a combination of manual and automated techniques to identify as many vulnerabilities as possible.

An audit can reveal the presence of vulnerabilities, but it cannot guarantee their absence. This report is a point-in-time assessment of the code at the commits named on the cover. It is not a warranty that the system is secure.

02 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	ACKNOWLEDGED
26	20	6

SEVERITY DISTRIBUTION

CLIENT	REPOSITORY	METHODS
Perpgame	github.com/perpgame/contracts	Manual review

CONTRACTS IN SCOPE

contracts/src/AgentCurve.sol	contracts/src/AgentTreasury.sol
contracts/src/TreasuryFactory.sol	contracts/src/TreasuryValuation.sol

TIMELINE · JUNE 2026

KICK-OFF	END OF AUDIT	REMEDIATIONS START	REMEDIATIONS END
09.06.2026	12.06.2026	12.06.2026	16.06.2026

STARTING COMMIT HASH	34c47cb45268b823ca80e140d19c38a073948d0f
FINAL COMMIT HASH	7dcfba6df0294384dd685bee16539b3e0565852e

03 PROJECT SUMMARY

What Perpgame is

Perpgame lets users buy into a launched token by paying USDC, minting ERC-20 shares priced off the net asset value of a basket of Hyperliquid leveraged tokens.

Sellers burn shares to redeem their pro-rata slice of the basket back to USDC, and a rebalancer sets target weights and rebalances the basket through Bounce.tech. The platform takes a 1% fee on each trade, and a scaling buy premium accrues to existing holders.

SECURITY POSTURE

BEFORE REMEDIATION 34c47cb

Mis-timed NAV and rate sampling

M-01 · M-02 · M-03 · M-04 · M-07 · M-08 · L-01

Rebalances and the bonding curve sized trades against stale pre-fee or pre-checkpoint values, so routine rebalances reverted or drifted off target. Redeemed USDC was re-deployed by weight on the next buy, and the curve priced whole buys at `supplyBefore`.

Single-leg freeze and DoS surfaces

M-05 · M-09 · M-10 · M-11 · L-06 · L-13

One unpriceable LT froze every NAV read, one stuck Bounce redemption halted all buys and sells, and unbounded scans opened a DoS.

Shared-NAV value transfer between holders

H-01 · M-12 · L-04 · L-05 · L-08 · L-11 · L-12

A duplicate LT under two symbols double-counted NAV so the rebalancer could drain co-investors, and idle-first payouts overpaid the first exiter. Near-confiscatory premiums, zero-share mints, and skipped fees on unredeemable legs shifted value between holders.

Thresholds and slippage floors

M-06 · L-02 · L-03 · L-07 · L-09 · L-10

Deploy thresholds and slippage floors did not track Bounce's live minimums. A small target weight could strand buyer capital idle, the async redeem path settled with no minimum, and per-leg floors reverted whole buys and rebalances.

AFTER REMEDIATION 7dcfba6

Twenty of twenty-six findings fixed. Several patches went beyond the literal bug: a separate `depositIdleUsdc` balance keeps redemption proceeds out of weight-based deploys, a recovery surface (`cancelRedeem`, `sweepDust`, `migrateLt`) unsticks stranded legs, and a factory-wide `paused` stop plus a `MAX_ASSETS` cap close the freeze and DoS class.

The six acknowledged items (M-04, M-06, L-02, L-07, L-08, L-09) are documented economic or operational limitations, none of them loss-of-funds. A 1% fee and a `returnLts` sell option landed after this fix set, run one regression pass over the merged branch before mainnet.

04 RECOMMENDATIONS

Beyond this audit

Three measures that extend the protection of this review after launch, independent of any individual finding.

R-01	**Bug bounty**	Stand up a public bounty on Immunefi or HackenProof with the top tier covering fund loss.
R-02	**Invariant monitoring**	Run an off-chain watcher that reconciles each treasury against its invariants and pages on divergence: `nav()` equal to the sum of each leg's `lt.ltToBaseAmount(balanceOf)` plus idle USDC, `depositIdleUsdc` never above the treasury's USDC balance, and `rebalanceInFlight` true only while at least one leg holds `userCredit(treasury) > 0`.
R-03	**Key hygiene**	The TreasuryFactory owner deploys every treasury, flips the global `paused` stop, and can call `rescue()`, so it is the highest-value key and belongs on a multisig with hardware signers. Each treasury's rebalancer sets weights and executes rebalances and should use hardware custody with a documented rotation and incident runbook.

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
`contracts/src/TreasuryFactory.sol`	Owner-only factory that CREATE2-deploys each token's treasury proxy, funded by a USDC permit. Holds the global `paused` kill switch and a `rescue()`, with `Ownable2Step` owner handover.
`contracts/src/AgentTreasury.sol`	Holds the basket's Bounce LTs and runs deploy, withdraw, and rebalance under a single rebalancer role. Upgradeable behind a beacon, capped at `MAX_ASSETS = 20` legs, and gated by the factory's global pause.
`contracts/src/TreasuryValuation.sol`	Stateless library holding the treasury's read-only valuation helpers, delegatecall-linked solely to keep the implementation under the EIP-170 code-size limit.
`contracts/src/AgentCurve.sol`	ERC-20 share token with a NAV-anchored bonding curve, minting at `nav / supply` times a premium capped by `MAX_EXTRA_PREMIUM`, with a 1% fee on every buy and sell.

PRIVILEGED ROLES

TreasuryFactory owner. Deploys treasuries, flips the global `paused` stop, and can `rescue()` factory-held tokens. Platform-held, belongs on a multisig with hardware signers.

Rebalancer. Per-treasury key that sets target weights and executes rebalance steps. Held by the platform agent for agentic tokens and by the creator for classic ones, with two-step handover via `proposeRebalancer` / `acceptRebalancer`.

CREATOR. Receives dust and stranded LT balances swept through `sweepDust` and `migrateLt`. Set to the token creator.

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
H-01	Duplicate LT across symbols double-counts NAV and lets the rebalancer drain co-investors	HIGH	FIXED
M-01	Async-redeemed rebalance USDC is re-deployed by weight on the next buy, undoing the rebalance	MEDIUM	FIXED
M-02	Deploy remainder reverts most of the buys	MEDIUM	FIXED
M-03	Rebalance buy sizes targets against a stale `navAtStart` after redeems change the rates	MEDIUM	FIXED
M-04	Buy premium is priced at `supplyBefore`, so one large buy pays less than many small ones	MEDIUM	ACKNOWLEDGED
M-05	One stuck Bounce redemption freezes all buys and sells for the whole fund	MEDIUM	FIXED
M-06	Deploy at low `minBps` leaves buyer capital undeployed as sells drain idle below the threshold	MEDIUM	ACKNOWLEDGED
M-07	Normal rebalances may revert	MEDIUM	FIXED
M-08	`executeRebalanceStep` sell path always triggers `lt.redeem` due to a wrong comparison	MEDIUM	FIXED
M-09	`_isFactoryLt` scan over the LT list scales with the Bounce catalog and can DoS portfolio configuration	MEDIUM	FIXED
M-10	A single held LT's reverting price view freezes the entire treasury with no on-chain recovery	MEDIUM	FIXED
M-11	`symbols` array grows without bound	MEDIUM	FIXED
M-12	Unredeemable sell leg: fee skipped or forfeited	MEDIUM	FIXED

06 FINDINGS

CONTINUED

ID	TITLE	SEVERITY	STATUS
L-01	Rebalance slippage check uses a pre-checkpoint rate that overstates redeemable value	LOW	FIXED
L-02	`prepareRedeem` path bypasses the `minRedeemUsdc` slippage check	LOW	ACKNOWLEDGED
L-03	`minLtMintUsdc` manually mirrors Bounce's `minTransactionSize`, reverting deploys until the rebalancer catches up	LOW	FIXED
L-04	`MAX_EXTRA_PREMIUM` allows a 99% buyer value transfer	LOW	FIXED
L-05	`withdrawLtsTo` overpays the first seller during the streaming-fee window	LOW	FIXED
L-06	Missing protocol-wide pause across treasuries	LOW	FIXED
L-07	Per-leg slippage floors make multi-leg buys and rebalances revert on a single unstable leg	LOW	ACKNOWLEDGED
L-08	`sell()` payout can leave small sellers with LT dust	LOW	ACKNOWLEDGED
L-09	`buy()` lets the caller waive mint slippage on a deploy of the entire shared idle pool	LOW	ACKNOWLEDGED
L-10	Rebalance is limited by Bounce's minimum redeem transaction size	LOW	FIXED
L-11	`buy` can accept payment yet mint zero shares	LOW	FIXED
L-12	Paused-leg skip in `_mintLTs` silently ignores the buyer's per-leg `minLtOuts` floor	LOW	FIXED
L-13	Fee is pushed to a hardcoded, unchangeable recipient	LOW	FIXED

HIGH SEVERITY · 1 FINDING

H-01 HIGH FIXED

Duplicate LT across symbols double-counts NAV and lets the rebalancer drain co-investors

The duplicate guard in `_setTargetPortfolio` only compares entries inside the array passed in the current call. It never checks a new symbol's `lt` against symbols already stored from previous calls, so the same LT can back two live symbols.

```
for (uint256 j = 0; j < i; j++) {
    if (keccak256(bytes(spec.symbol)) == keccak256(bytes(newPortfolio[j].symbol))) {
        revert DuplicateSymbol(spec.symbol);
    }
    if (spec.lt == newPortfolio[j].lt) revert DuplicateLt(spec.lt);   // only checks within
this array
}
```

If `ltX` is already registered under symbol `"A"`, a second call with a single entry binding `"B"` to the same `ltX` slips through, and `"B"` is appended pointing at the same LT. The old `"A"` is zeroed to `bps = 0` but `_pruneExitedSymbols` keeps it, because it only removes a symbol whose LT balance is zero.

`nav()` then walks `symbols[]` and reads `balanceOf` once per symbol, so the one real balance is counted twice:

```
for (uint256 i = 0; i < n; i++) {
    IBounceLT lt = IBounceLT(assets[symbols[i]].lt);
    uint256 bal = lt.balanceOf(address(this));
    if (bal > 0) total += lt.ltToBaseAmount(bal);
}
```

`withdrawLtsTo` walks the same array on a sell, so the inflated NAV inflates the seller's payout and the per-symbol loop transfers LT out of the same balance more than once. Each extra call adds one more symbol on the same LT, so NAV can be inflated to an arbitrary N times the truth. This breaks two whitepaper invariants, "duplicates are rejected" and "a compromised rebalancer cannot steal principal". In classic mode the on-chain rebalancer is the token creator, an untrusted party, while the curve is open to public buyers.

1. Genesis registers `"A"` pointing at `ltX` at 100 percent, the creator holds 1000e18 shares.
2. A public buyer buys in, supply is 2000e18, the treasury holds `ltX` worth 1000 USDC, both own 50 percent.
3. The creator calls `setTargetPortfolio` with a single entry binding `"B"` to `ltX`, and the within-array guard lets it pass.
4. `nav()` now counts `ltX` twice and reports 2000 USDC for 1000 USDC of real backing.
5. The creator sells their 50 percent, the payout loop sends `ltX` twice, so they take 750e18 instead of the fair 500e18, leaving 250e18 for the buyer who still holds 50 percent.

IMPACT

A token creator can inflate `nav()` and drain the principal of public buyers in their own curve, approaching 100 percent of the treasury with enough duplicate registrations. Severity assumes the creator is not trusted toward co-investors who buy into the public curve, consistent with the stated invariant.

RECOMMENDATION

Enforce that each LT appears in `symbols[]` at most once. The duplicate guard must check the persistent set, not just the array passed in the current call: keep a reverse index and clear it on prune.

```
+    /// every LT address currently bound to a registered symbol
+    mapping(address ⇒ bool) private ltRegistered;
@@ _setTargetPortfolio — the new-symbol branch
         if (!asset.registered) {
             if (!_isFactoryLt(factoryLts, spec.lt)) revert LtNotAllowed(spec.lt);
+            if (ltRegistered[spec.lt]) revert DuplicateLt(spec.lt);
+            ltRegistered[spec.lt] = true;
             symbols.push(spec.symbol);
             assets[spec.symbol] = AssetConfig({lt: spec.lt, targetBps: spec.bps, registered:
true});
         } else if (asset.lt ≠ spec.lt) {
@@ _pruneExitedSymbols — when a symbol is removed
+            ltRegistered[a.lt] = false;
             symbols.pop();
             delete assets[sym];
```

MEDIUM SEVERITY · 12 FINDINGS

M-01 MEDIUM FIXED

Async-redeemed rebalance USDC is re-deployed by weight on the next buy, undoing the rebalance

When a rebalance shrink goes through `prepareRedeem`, the redeemed USDC arrives as idle only after Bounce settles, and the next `buy` deploys that idle through `_mintLTs`, which splits it across legs by target weight, not toward the rebalance's target balances. The settled funds re-inflate the over-weight leg instead of topping up the under-weight one.

```
// AgentTreasury.sol — _mintLTs (reached via buy → _deployIdle), deploys ALL idle by weight
usdcToAllocate = (usdcAmount * uint256(assets[sym].targetBps)) / BPS_DENOM;
```

`executeRebalanceStep` cannot place the funds itself in this case: the grow leg is deferred while settlement is pending.

```
// AgentTreasury.sol — executeRebalanceStep buy loop
if (growUsdc > usdcBal) {
    if (rebalanceInFlight) { emit MintDeferredForSettlement(symbols[i], growUsdc, usdcBal);
continue; }
    ...
}
```

Once the flag clears, the idle is up for grabs: a `buy` deploys it by weight via `_deployIdle`, whereas only a fresh `executeRebalanceStep` would deploy it toward targets. Two legs, target 90/10, holding 29,000 / 1,000 against a target of 27,000 / 3,000:

1. `executeRebalanceStep` redeems 2,000 from leg 1 via `prepareRedeem`. Leg 2's grow is deferred and the flag is set.
2. Settlement delivers about 2,000 idle and `settleRebalance` clears the flag.
3. A `buy` deploys the idle 90/10: leg 1 gains 1,800 and leg 2 gains 200, ending at 28,800 / 1,200, roughly 96/4, back to the original imbalance.

IMPACT

Any async-redemption rebalance can be silently undone by a single intervening buy, so the rebalancer cannot reliably converge to target weights, and each redeem-then-redeploy cycle burns the Bounce redemption fee for no net rebalancing. Whether a rebalance works at all depends on the redeem path and who calls next.

RECOMMENDATION

Keep async-redemption proceeds out of the weight-based buy deploy: reserve the in-flight redeemed amount so `_deployIdle` cannot consume it, and deploy it only through the balance-aware rebalance path. Settled rebalance funds then land on the under-weight legs regardless of who transacts next.

M-02 MEDIUM FIXED

Deploy remainder reverts most of the buys

`_mintLTs` assigns the leftover USDC to the last `symbols[]` entry without checking that leg's `targetBps`, so when the last entry is a zeroed leg pointing at a live LT it receives only rounding dust, and `lt.mint` reverts on `minTransactionSize`.

```
// AgentTreasury.sol — _mintLTs
if (i == n - 1) {
    usdcToAllocate = usdcAmount - deployed;   // remainder, regardless of this leg bps
} else {
    usdcToAllocate = (usdcAmount * uint256(assets[sym].targetBps)) / BPS_DENOM;
    deployed += usdcToAllocate;
}
if (usdcToAllocate == 0) {                    // dust is NOT zero, so this does not catch it
    if (minLtOuts[i] != 0) revert SlippageExceeded();
    continue;
}
...
uint256 ltOut = lt.mint(address(this), usdcToAllocate, minLtOuts[i]);  // reverts if dust <
minTransactionSize
```

Active weights sum to `BPS_DENOM`, so the active legs at non-last indices consume nearly the whole `usdcAmount`. A zeroed leg (`targetBps == 0`) that persists in `symbols[]` and lands at index `n-1` then gets a few wei of flooring dust. That dust is non-zero, so the `usdcToAllocate == 0` guard skips it, and `lt.mint` reverts with `BelowMinTransactionSize`. `minDeployUsdc()` does not prevent this: it is derived from `minBps`, the smallest non-zero weight, and never accounts for the remainder landing on a zero-weight leg.

IMPACT

Every deploy-triggering buy reverts while a `targetBps == 0` symbol with a live, non-paused LT occupies the last slot of `symbols[]`, a state reachable through normal basket rotation, since zeroed legs are not removed from `symbols[]`. Buys stay bricked until the array order changes.

RECOMMENDATION

Treat the remainder leg like any other: skip it when its `targetBps == 0`, and route the rounding remainder to idle.

M-03 MEDIUM FIXED

Rebalance buy sizes targets against a stale `navAtStart` after redeems change the rates

`executeRebalanceStep` snapshots `navAtStart` once at the top, then uses it to compute `targetUsdcValue` for every leg. The sell loop that runs in between calls `redeem` / `prepareRedeem`, which pay redemption fees out of the LT and trigger Bounce's `_checkpoint` streaming fee. Both permanently lower the true nav and the per-LT `exchangeRate`, so the buy loop targets a nav that no longer exists.

```
// AgentTreasury.sol — executeRebalanceStep
uint256 navAtStart = nav();                                    // snapshot: pre-fee, un-checkpointed rates
// sell loop: each redeem/prepareRedeem pays a redemption fee out of the LT
// and runs _checkpoint(), which streams a fee out of totalAssets → exchangeRate drops
uint256 actualBaseOut = lt.redeem(address(this), shrinkLt, minRedeemUsdc[i]);
...
// buy loop:
uint256 currentUsdcValue = lt.ltToBaseAmount(lt.balanceOf(address(this)));  // fresh, LOWER rates
uint256 targetUsdcValue = (navAtStart * uint256(assets[symbols[i]].targetBps)) / BPS_DENOM;
// stale, HIGHER nav
uint256 growUsdc = targetUsdcValue - currentUsdcValue;
```

In Bounce, `redeem` removes gross value proportional to burned supply, but the redemption fee leaves the contract and `_checkpoint` streams a fee out of `totalAssets` without burning supply, so `exchangeRate = totalAssets / totalSupply` falls. By the buy loop the portfolio is genuinely worth less than `navAtStart` while each leg's `targetUsdcValue` is still a fraction of the inflated snapshot.

IMPACT

The buy loop over-sizes grow targets: it allocates fractions of a nav that has already shrunk by the fees realized during selling, so legs it processes are minted past their true target weight and the portfolio drifts from the intended `targetBps` proportions.

RECOMMENDATION

Recompute the nav reference after the sell loop completes, so both `targetUsdcValue` and `currentUsdcValue` are measured against the same post-redeem rates.

M-04 MEDIUM ACKNOWLEDGED

Buy premium is priced at `supplyBefore`, so one large buy pays less than many small ones

`AgentCurve::buy` charges the premium using `premium(supplyBefore)`, a single value read from the supply before the buy, and applies it to the whole `usdcIn`, no matter how far that one buy pushes the supply.

```
// AgentCurve.sol -- buy / _quoteBuy
uint256 supplyBefore = totalSupply();
uint256 p = premium(supplyBefore); // one premium for the entire buy
return (usdcIn * supplyBefore * ONE) / (navBefore * p);
```

`premium()` rises as supply approaches `PREMIUM_CAP_SUPPLY`, so each AGENT minted closer to the cap should cost more. Because the premium is fixed at the starting supply, one buy that jumps the whole distance pays the low starting premium on every AGENT, while small steps pay the premium that rises under their feet.

Both buyers push supply from 950 to the cap at 1,000, minting 50 AGENT at a fair pre-premium price of 1 USDC each. What each AGENT costs depends on the supply at the moment it is minted:

AGENT MINTED AT SUPPLY	`premium(supply)` THERE	ONE LARGE BUY CHARGES	50 SMALL BUYS CHARGE
950 (start)	91.25x	91.25x	91.25x
975 (halfway)	96.1x	91.25x	96.1x
999 (last one)	100.8x	91.25x	100.8x
Total for all 50 AGENT		**~4,562 USDC**	**~4,799 USDC**

The large buy freezes the premium at 91.25x for every AGENT, even the ones minted right at the cap. A buyer taking the entire cap in one transaction would pay almost no premium.

IMPACT

The premium curve charges late buyers more and routes the surplus to existing holders. A buyer who concentrates demand into one large transaction underpays versus incremental buyers, so existing holders collect less than the curve dictates.

RECOMMENDATION

The problem lies in the pricing formula itself. A virtual constant-product AMM makes the premium paid independent of trade size. At the very least, document the behavior.

M-05 MEDIUM FIXED

One stuck Bounce redemption freezes all buys and sells for the whole fund

When an LT cannot be redeemed atomically, `executeRebalanceStep` calls `lt.prepareRedeem`, which escrows the LT and records `userCredit` on that LT, and sets a single global `rebalanceInFlight = true`. `deployUsdc` and `withdrawLtsTo` both revert with `RebalancePending` while the flag is set, and it clears only via `_clearInFlightIfSettled`, which stays set while any held LT still has `userCredit(treasury) > 0`.

On the Bounce side redemptions are per-LT, and `_executeRedemption` has two conditions that each return early without zeroing `userCredit`. The second is the problem: a credit too small to pay the flat redemption fee never executes. The LT executor may also, intentionally or not, simply fail to execute the withdrawal.

```
if (baseAssetBalance() < baseAmount_) return false;
...
if (baseAmount_ < redemptionFee_) return false;
```

The flag is global and keyed on `userCredit`, so one stuck leg blocks buys and sells for the whole basket, even legs Bounce already settled. A price drop never zeroes `userCredit`, so the treasury can hold almost none of a leg yet stay frozen by its dust credit. There is no `cancelRedeem` path and no emergency exit, and `settleRebalance()` just re-checks `userCredit > 0`.

RECOMMENDATION

Add a function that calls `cancelRedeem()` on a pending leg after Bounce's 1h delay. It pulls the possibly depreciated LT back into the treasury's balance, zeroes `userCredit`, and lets `_clearInFlightIfSettled` clear the flag.

```
+    function cancelRedeem(string calldata symbol) external onlyRebalancer {
+        IBounceLT(assets[symbol].lt).cancelRedeem();
+        _clearInFlightIfSettled();
+    }
```

M-06 MEDIUM ACKNOWLEDGED

Deploy at low `minBps` leaves buyer capital undeployed as sells drain idle below the threshold

`_deployIdle` defers the entire deploy whenever idle USDC is below `minDeployUsdc()`, and that threshold scales inversely with `minBps`, the smallest non-zero target weight. Because `withdrawLtsTo` pays sells out of idle first, idle is continuously drained and rarely climbs to a threshold set high by a small weight, so buyer USDC is minted into LTs only sporadically, if ever.

```
// AgentTreasury.sol — _deployIdle
uint256 idle = USDC.balanceOf(address(this));
uint256 threshold = minDeployUsdc();
if (idle < threshold) {
    emit DeployDeferred(idle, threshold);
    return;            // ALL idle stays undeployed, not just the sub-min remainder
}
_mintLTs(idle, minLtOuts);
```

```
// AgentTreasury.sol — minDeployUsdc: threshold grows as minBps shrinks
return (minLtMintUsdc * BPS_DENOM + uint256(minBps) - 1) / uint256(minBps);
```

Every sell pulls USDC straight out of idle before touching LTs, so idle is pushed back down between buys. With `minLtMintUsdc = 10e6`, the smallest weight the rebalancer assigns sets how much idle must accumulate before any deploy happens:

SMALLEST TARGET WEIGHT (`minBps`)	`minDeployUsdc`
1000 (10%)	100 USDC
100 (1%)	1,000 USDC
10 (0.1%)	10,000 USDC

At a 0.1% leg, idle must reach 10,000 USDC and survive intervening sells before a single deploy fires. Under two-sided flow the threshold is rarely crossed, so capital stays idle.

IMPACT

The protocol's core function, deploying buyer capital into the target leveraged-token portfolio, is effectively disabled when the rebalancer assigns any small but legitimate weight. Buyers receive AGENT backed by idle USDC at face value rather than the intended leveraged exposure, and `nav` drifts from the target portfolio. No funds are lost, but a rebalancer can trigger this silently by shrinking one leg's weight.

RECOMMENDATION

Stop gating the whole deploy on a single global threshold. Deploy per leg: mint every symbol whose computed allocation meets the per-leg minimum and leave only the sub-minimum legs in idle, which the rebalancer can rebalance later. At the very least, document how the smallest weight drives the deploy threshold so the rebalancer understands the effect.

M-07 MEDIUM FIXED

Normal rebalances may revert

A rebalance sells one token to buy another. It plans the buy from the portfolio value before the redemption fee, but receives cash after the fee, so it is short by exactly the fee and reverts. Treasury holds 2000 USDC fully invested, 1000 in A and 1000 in B:

1. Goal: re-weight 50/50 to 20/80, so sell 600 of A and buy 600 of B in one transaction, sell loop then buy loop.
2. Selling 600 of A pays Bounce's 1.5% fee of 9 USDC, so the treasury receives 591, not 600.
3. The buy step still wants 600, sized up front from full NAV, but only 591 is on hand.
4. The whole transaction reverts with `InsufficientUsdcForMint(600e6, 591e6)`. Nothing moves.

The buy amount is sized from value before the fee, but the cash that arrives is after the fee, and the gap is exactly the fee. With no idle cash, every sell-A-to-fund-B rebalance reverts.

IMPACT

Rebalancing is the protocol's core operation, and the natural parameters an off-chain agent emits revert whenever idle USDC is near zero, the normal state. The caller cannot reliably pre-set a cap, because the shortfall depends on `redemptionFee * targetLeverage` and the live `exchangeRate`.

RECOMMENDATION

On the atomic path, mint what the proceeds actually cover, mirroring the deferred branch, instead of reverting:

```
  if (growUsdc > usdcBal) {
      if (rebalanceInFlight) { emit MintDeferredForSettlement(...); continue; }
-     revert InsufficientUsdcForMint(symbols[i], growUsdc, usdcBal);
+     if (usdcBal == 0) continue;   // nothing realized for this leg this step
+     growUsdc = usdcBal;           // buy the realized net; don't over-target the fee
  }
```

M-08 MEDIUM FIXED

`executeRebalanceStep` sell path always triggers `lt.redeem` due to a wrong comparison

In the rebalance sell loop, the treasury decides whether to redeem atomically or queue it with `prepareRedeem`:

```
uint256 expectedBaseOut = lt.ltToBaseAmount(shrinkLt);
if (expectedBaseOut < minRedeemUsdc[i]) revert SlippageExceeded();
// it LT has enough USDC in buffer, use atomic redeem(), else async path
int256 bufferRaw = LT_HELPER.getLeveragedTokenBufferAssetValue(address(lt));
uint256 buffer = bufferRaw < 0 ? 0 : uint256(bufferRaw);
if (expectedBaseOut ⩽ buffer) {
    uint256 actualBaseOut = lt.redeem(address(this), shrinkLt, minRedeemUsdc[i]);
    emit AtomicRedeem(symbols[i], address(lt), shrinkLt, actualBaseOut);
} else {
```

The two operands are on different scales. `expectedBaseOut = lt.ltToBaseAmount(shrinkLt)` is 6-decimal USDC, because the real Bounce `ltToBaseAmount` ends in `scaleTo(6)`, while `buffer` comes from `getLeveragedTokenBufferAssetValue`, which is 18-decimal via `baseAssetBalance().scaleFrom(6)`. The guard compares them directly with no rescaling, so `buffer` is about 1e12 times too large and `expectedBaseOut ⩽ buffer` is essentially always true. The atomic branch is always taken and the async `prepareRedeem` fallback is dead code.

The treasury then calls `lt.redeem()`, which reverts when the requested amount exceeds the LT's idle EVM-side USDC:

```
uint256 baseAmount_ = ltToBaseAmount(ltAmount_);
if (baseAmount_ > baseAssetBalance()) revert InsufficientBalance();
```

RECOMMENDATION

Scale `expectedBaseOut` up from 6-decimal USDC to 1e18 before comparing, or scale `buffer` down to 6:

```
+    /// USDC (6-dec) to 1e18 scale, matching the buffer helper's scaleFrom(6).
+    uint256 private constant USDC_TO_WAD = 1e12;
```

```
             int256 bufferRaw = LT_HELPER.getLeveragedTokenBufferAssetValue(address(lt));
             uint256 buffer = bufferRaw < 0 ? 0 : uint256(bufferRaw);
-            if (expectedBaseOut ⩽ buffer) {
+            if (expectedBaseOut * USDC_TO_WAD ⩽ buffer) {
```

M-09 MEDIUM FIXED

`_isFactoryLt` scan over the LT list scales with the Bounce catalog and can DoS portfolio configuration

`_setTargetPortfolio` validates each newly-registered spec by copying the entire `BOUNCE_FACTORY.lts()` array into memory and linear-scanning it via `_isFactoryLt`, even though Bounce's `Factory` exposes an O(1) `ltExists(address)` mapping returning the same boolean.

```
// contracts/src/AgentTreasury.sol — _setTargetPortfolio (excerpt)
address[] memory factoryLts = BOUNCE_FACTORY.lts();   // O(N_bounce) SLOADs every call
for (uint256 i = 0; i < newPortfolio.length; i++) {
    ...
    if (!asset.registered) {
        if (!_isFactoryLt(factoryLts, spec.lt)) revert LtNotAllowed(spec.lt);
        ...
    }
}
```

IMPACT

As Bounce's catalog grows, both `initialize` and `setTargetPortfolio` hit a gas ceiling that Perpgame does not control, a DoS on portfolio configuration in the worst case.

RECOMMENDATION

Add `ltExists` to the local interface and use it instead of the linear scan. `_isFactoryLt` becomes dead code and can be removed.

```
 interface IBounceFactory {
     function lts() external view returns (address[] memory);
+    function ltExists(address lt) external view returns (bool);
 }
```

```
-    address[] memory factoryLts = BOUNCE_FACTORY.lts();
     for (uint256 i = 0; i < newPortfolio.length; i++) {
         AssetSpec memory spec = newPortfolio[i];
         ...
         if (!asset.registered) {
-            if (!_isFactoryLt(factoryLts, spec.lt)) revert LtNotAllowed(spec.lt);
+            if (!BOUNCE_FACTORY.ltExists(spec.lt)) revert LtNotAllowed(spec.lt);
             ...
         }
     }
```

M-10 MEDIUM FIXED

A single held LT's reverting price view freezes the entire treasury with no on-chain recovery

`nav()` is the only pricing primitive and it walks `symbols[]` calling each leg.

```
function nav() public view returns (uint256 total) {
    uint256 n = symbols.length;
    for (uint256 i = 0; i < n; i++) {
        IBounceLT lt = IBounceLT(assets[symbols[i]].lt);
        uint256 bal = lt.balanceOf(address(this));
        if (bal > 0) total += lt.ltToBaseAmount(bal);   // no try/catch
    }
    total += USDC.balanceOf(address(this));
}
```

`nav()` is called on every path, so if one leg's `ltToBaseAmount` reverts, all of them revert. On HyperEVM mainnet a Bounce LT prices itself off HyperCore through a read precompile, and the HyperCore docs state that precompiles called on invalid inputs, such as invalid assets, return an error and consume all gas. If the LT's underlying HyperCore token is delisted or its index becomes invalid, `ltToBaseAmount` reverts permanently.

There is no on-chain recovery. `_setTargetPortfolio` emits `RebalanceTargetSet(..., nav(), ...)` and `executeRebalanceStep` starts with `navAtStart = nav()`, so the rebalancer can neither rotate the bad LT out nor rebalance. The treasury has no rescue or skim and the LT cannot be moved out by any other path, so the only fix is a 48h-timelocked beacon-implementation upgrade.

1. A treasury holds `HYPE5L`. Buys, sells, and rebalances all work normally.
2. Hyperliquid delists the LT's underlying asset, so the `spotBalance` precompile returns an error, `HYPE5L.ltToBaseAmount(bal)` reverts, and `nav()` reverts with it.
3. Every holder is locked out: `buy` reverts at `navBefore`, `sell` reverts inside `withdrawLtsTo`.
4. The rebalancer's removal attempt reverts at the `RebalanceTargetSet(..., nav(), ...)` emit, and `executeRebalanceStep` reverts at `navAtStart = nav()`.
5. Funds are frozen until a 48h-timelocked beacon upgrade ships a patched implementation.

IMPACT

A single delisted or unpriceable held LT freezes the entire treasury: no holder can enter or exit, and the rebalancer cannot remove the offending leg or rebalance. Recovery requires a 48h-timelocked beacon upgrade by the multisig.

RECOMMENDATION

This is a rare, high-impact, recoverable event, so a minimal fix is reasonable. Drop the `nav()` call from the `RebalanceTargetSet` emit so portfolio rotation is never bricked. That alone does not unfreeze `buy` / `sell` or remove the stuck leg, so if no-upgrade recovery is wanted, add a guardian `forceRemoveLt(symbol)` that evicts an unpriceable leg without pricing it, or a trustless in-kind `emergencyExit` paying `ltBalance * agentShares / totalSupply` per leg with no `ltToBaseAmount` call.

M-11 MEDIUM FIXED

`symbols` array grows without bound

`AgentTreasury` never reliably shrinks `symbols[]`: `_setTargetPortfolio` only zeroes a dropped leg's `targetBps`, and the sole removal path `_pruneExitedSymbols` deletes a symbol only when its LT balance and `userCredit` are both zero, a condition two independent mechanisms keep permanently false.

```
// AgentTreasury.sol — _setTargetPortfolio drops a leg by zeroing, not removing
for (uint256 i = 0; i < existingN; i++) {
    assets[symbols[i]].targetBps = 0;   // zeroed, still in symbols[]
}
// AgentTreasury.sol — _pruneExitedSymbols, the only removal path
if (a.targetBps == 0 && lt.balanceOf(address(this)) == 0 && lt.userCredit(address(this)) == 0)
{
    // ... swap-pop and delete
}
```

Bounce LTs are ordinary ERC-20s, so anyone can send 1 wei of a retired LT to keep `balanceOf ≠ 0` forever. A 1-wei leg also has `ltToBaseAmount == 0`, so the redeem and withdraw loops skip it and the balance never returns to zero. The contract has no sweep, so the prune condition can never be satisfied again.

A registered symbol's `lt` is also immutable, and `_setTargetPortfolio` reverts any change:

```
// AgentTreasury.sol — _setTargetPortfolio
} else if (asset.lt ≠ spec.lt) {
    revert SymbolTokenMismatch(spec.symbol, asset.lt, spec.lt);  // blocks the refresh
}
```

Bounce's `Factory::redeployLt` legitimately swaps an LT's contract address while preserving its deterministic symbol. After such a redeploy the rebalancer cannot re-point the existing entry and must register a fresh symbol for the same product. The original entry orphans with a balance the treasury can no longer drain through Bounce, so it is unprunable. Every path then iterates the full stored `symbols[]`, each dead entry costing an SLOAD plus an external call, and `deployUsdc` reverts with `LengthMismatch` unless `minLtOuts.length == symbols.length`, so every buyer must pad zeros for every dead leg and the rebalancer must pass four such arrays.

IMPACT

`symbols[]` expands monotonically under two forces the protocol cannot counter. Every user permanently overpays gas proportional to the dead legs, every buy and rebalance must carry full-length slippage arrays covering legs that no longer exist, and in the worst case most protocol functions hard-DoS.

RECOMMENDATION

1. Add a `sweepDust` so retired LTs become deletable.
2. Add a rebalancer-callable migration that updates `assets[symbol].lt` in place, so a Bounce redeploy refreshes the existing symbol instead of orphaning it.

M-12 MEDIUM FIXED

Unredeemable sell leg: fee skipped or forfeited

On a sell, when a leg cannot be instant-redeemed, `withdrawLtsTo` branches on `returnLts`, and the 1% fee is taken only on the USDC that actually redeemed:

```
} catch { if (returnLts) IERC20(address(lt)).safeTransfer(recipient, ltOut); }  // else: leg left in treasury
...
uint256 fee = (usdcOut * SELL_FEE_BPS) / BPS_DENOM;   // fee only on redeemed USDC
```

IMPACT

With `returnLts = true` the LT goes to the seller untaxed, so the 1% on that leg is not collected, and if nothing redeems the fee is skipped entirely. With `returnLts = false` the leg is skipped and stays in the treasury: the seller burned all their shares but is paid only for what redeemed, so the rest effectively passes to the other holders, and since `quoteSellUsdc` excludes that leg the shortfall is easy to miss (seller receives ~$495 with ~$500 left behind). Real instant buffers are small, ETH5L holds about $198, so this is the common path on a larger sell rather than a rare edge.

RECOMMENDATION

In the `catch`, always hand the seller the LT for the unredeemable part, minus a 1% fee in LT, regardless of `returnLts`. This covers both cases:

```
} catch {
    uint256 ltFee = (ltOut * SELL_FEE_BPS) / BPS_DENOM;
    if (ltFee > 0) IERC20(address(lt)).safeTransfer(FEE_RECIPIENT, ltFee);
    IERC20(address(lt)).safeTransfer(recipient, ltOut - ltFee);
}
```

LOW SEVERITY · 13 FINDINGS

L-01 LOW FIXED

Rebalance slippage check uses a pre-checkpoint rate that overstates redeemable value

`executeRebalanceStep` checks `minRedeemUsdc[i]` against `expectedBaseOut = lt.ltToBaseAmount(shrinkLt)`, computed at the current `exchangeRate()`. But `redeem` and `prepareRedeem` each run `_checkpoint()` first, which streams the management fee out of `totalAssets` and lowers `exchangeRate`, so the value the redeem actually computes is smaller than `expectedBaseOut`.

```
// AgentTreasury.sol — executeRebalanceStep
uint256 expectedBaseOut = lt.ltToBaseAmount(shrinkLt);          // pre-checkpoint rate
if (expectedBaseOut < minRedeemUsdc[i]) revert SlippageExceeded();
```

```
// LeveragedToken.sol — redeem
_checkpoint();                                                  // streams fee, exchangeRate drops
uint256 baseAmount_ = ltToBaseAmount(ltAmount_);               // recomputed at the LOWER rate
```

RECOMMENDATION

Run the slippage check against the actually redeemed base amount.

L-02 LOW ACKNOWLEDGED

`prepareRedeem` path bypasses the `minRedeemUsdc` slippage check

`executeRebalanceStep` validates `minRedeemUsdc[i]` against `expectedBaseOut` once, then on the async branch calls `prepareRedeem`, which carries no minimum and settles later at a different rate, so the check guards nothing for that path.

```
// AgentTreasury.sol — executeRebalanceStep sell loop
uint256 expectedBaseOut = lt.ltToBaseAmount(shrinkLt);
if (expectedBaseOut < minRedeemUsdc[i]) revert SlippageExceeded();   // checked at current rate
...
if (expectedBaseOut ⩽ buffer) {
    lt.redeem(address(this), shrinkLt, minRedeemUsdc[i]);            // atomic: min enforced on realized output
} else {
    lt.prepareRedeem(shrinkLt);                                      // async: no min, settles later at unknown rate
}
```

`prepareRedeem` only records `userCredit`. The USDC payout happens through Bounce's external keeper settlement at whatever `exchangeRate()` prevails then, and nothing ties `minRedeemUsdc[i]` to that settlement.

IMPACT

If `exchangeRate()` falls between `prepareRedeem` and settlement, the treasury receives less than `minRedeemUsdc[i]` and nothing reverts or flags it.

RECOMMENDATION

A true atomic revert is not possible since settlement is external. Document the behaviour, or skip the `prepareRedeem` step entirely and redeem only the LTs available for instant redemption.

L-03 LOW FIXED

`minLtMintUsdc` manually mirrors Bounce's `minTransactionSize`, reverting deploys until the rebalancer catches up

`minDeployUsdc` sizes the deploy threshold from `minLtMintUsdc`, a value the rebalancer must manually keep in sync with Bounce's `minTransactionSize`. Bounce can change `minTransactionSize` at any time, and until the rebalancer mirrors it, leg allocations sized against the stale `minLtMintUsdc` fall below Bounce's minimum and `lt.mint` reverts.

```
// AgentTreasury.sol — minLtMintUsdc is a hand-set mirror, not a live read
function setMinLtMintUsdc(uint256 next) external onlyRebalancer {
    if (next > MAX_MIN_LT_MINT_USDC) revert MinLtMintUsdcTooHigh(next, MAX_MIN_LT_MINT_USDC);
    minLtMintUsdc = next;
    ...
}
```

```
// LeveragedToken.sol — the live bound the deploy must clear
if (baseAmount_ < $.globalStorage.minTransactionSize()) revert BelowMinTransactionSize();
```

IMPACT

Buys and rebalance steps revert whenever Bounce raises `minTransactionSize` above the current `minLtMintUsdc`, and stay broken until the rebalancer notices and calls `setMinLtMintUsdc`, forcing continuous off-chain monitoring of a third party's parameter.

RECOMMENDATION

Read `minTransactionSize` from Bounce directly when sizing the deploy threshold, so the contract always tracks the live bound and no rebalancer action is required.

L-04 LOW FIXED

`MAX_EXTRA_PREMIUM` allows a 99% buyer value transfer

`AgentCurve` bounds the per-launch premium add-on at `MAX_EXTRA_PREMIUM = 100e18`, which permits a premium multiplier of 101x.

```
// AgentCurve.sol
uint256 private constant MAX_EXTRA_PREMIUM = 100e18;
...
if (extraPremium_ > MAX_EXTRA_PREMIUM) revert ExtraPremiumTooHigh(extraPremium_,
MAX_EXTRA_PREMIUM);
...
return (usdcIn * supplyBefore * ONE) / (navBefore * p); // p up to 101e18
```

At `EXTRA_PREMIUM = 100e18` and `supply ⩾ PREMIUM_CAP_SUPPLY`, a buyer mints `1/101` of the NAV-pro-rata amount: roughly 99% of every USDC deposit accrues to existing holders, and since sells settle at the NAV floor, the buyer can recover at most about 1% of it by exiting.

IMPACT

A launch configured at the cap is economically confiscatory for buyers past `PREMIUM_CAP_SUPPLY` while passing all constructor validation.

RECOMMENDATION

Lower `MAX_EXTRA_PREMIUM` to a bound consistent with a buy/sell spread, for example `1e18`, capping the premium at 2x.

L-05 LOW FIXED

`withdrawLtsTo` overpays the first seller during the streaming-fee window

Bounce LTs charge a streaming fee that accrues every second but is only deducted from `exchangeRate` at the next `mint` / `redeem` / `prepareRedeem` of that LT. Between those checkpoints `nav()` is stale-high by the size of the un-deducted fee. `withdrawLtsTo` sizes the seller's payout off that stale `nav()` and pays out of idle USDC first:

```
uint256 navTotal = idle + totalLtValue;             // totalLtValue uses the STALE rate
uint256 notional = (navTotal * agentShares) / totalShares;
...
uint256 usdcOut = notional ⩽ idle ? notional : idle;
if (usdcOut > 0) USDC.safeTransfer(recipient, usdcOut);
```

If `notional` fits in idle, the seller walks away with USDC at the overstated mark and zero LT. The treasury still holds the entire LT position, and when the checkpoint eventually fires, the streaming fee on the treasury's LT slice falls entirely on whoever is still holding curve shares. The in-kind branch does not leak: a seller who carries an LT slice out pays their share of the fee at their own later checkpoint. Resident idle is reachable without any privileged action: paused legs, buys below `minDeployUsdc`, async-rebalance settlement windows, and `bps = 0` retired legs all park USDC idle.

One-leg treasury, 5x long, 2%/yr streaming fee, so effective LT decay is 10%/yr:

1. Alice seeds 100k, fully deployed. The owner pauses the leg.
2. Bob buys 60k. The mint is skipped, 60k stays idle, `nav()` reads 160k, and Bob owns 1/6 of supply.
3. 30 days pass with no checkpoint on this LT. The treasury's 100k LT slice has silently accrued about 822 USDC of streaming fee.
4. Bob sells. `notional = 160k * 1/6 = 26,666.67` fits in idle, so he gets it all in USDC and zero LT.
5. A later mint or redeem fires `_checkpoint`. The 822 USDC fee falls entirely on Alice, while Bob's fair share would have been about 137.

On real Bounce bytecode Bob's excess equals Alice's deficit to 1 wei. External LT holders pay their own separate fee.

RECOMMENDATION

Either pay idle USDC pro-rata or document that the cash branch can extract stale-fee value from remaining holders.

```
- uint256 usdcOut = notional ⩽ idle ? notional : idle;
+ uint256 usdcOut = (idle * agentShares) / totalShares;
  if (usdcOut > 0) USDC.safeTransfer(recipient, usdcOut);
  uint256 remaining = notional - usdcOut;
```

L-06 LOW FIXED

Missing protocol-wide pause across treasuries

`AgentTreasury` has no pause mechanism. `TreasuryFactory` deploys an unbounded set of treasuries behind a shared beacon, but there is no operator-level control to halt them during an incident, for example an LT mispricing on Bounce, which directly corrupts `nav()` and therefore buy and sell pricing in every treasury. The only protocol-wide lever is a beacon implementation upgrade, and the only per-treasury lever is each rebalancer deliberately keeping a redemption in flight. Neither is a usable emergency stop.

RECOMMENDATION

Add a shared pause registry, for example a `paused()` flag on `TreasuryFactory` or a dedicated guardian contract referenced by the treasury implementation, and check it in `deployUsdc` and `withdrawLtsTo`.

L-07 LOW ACKNOWLEDGED

Per-leg slippage floors make multi-leg buys and rebalances revert on a single unstable leg

`_mintLTs` enforces a separate slippage floor for every LT leg via `minLtOuts[i]`, so a single leg breaching its floor reverts the entire buy, even though the sell path already protects users with one aggregate floor.

```
// AgentTreasury.sol — _mintLTs (buy/deploy path)
USDC.forceApprove(address(lt), usdcToAllocate);
uint256 ltOut = lt.mint(address(this), usdcToAllocate, minLtOuts[i]);
if (ltOut < minLtOuts[i]) revert SlippageExceeded(); // per-leg AND-gate
```

The same per-leg pattern gates `executeRebalanceStep` through `minRedeemUsdc[i]` and `minMintLt[i]`. Because these are leveraged tokens whose exchange rates move quickly, the buyer must set N floors that all hold simultaneously at execution time, and the chance of the whole transaction reverting grows with each added leg. By contrast, `withdrawLtsTo` guards the sell with a single aggregate bound and lets the legs net out against each other:

```
// AgentTreasury.sol — withdrawLtsTo (sell path)
uint256 notional = (navTotal * agentShares) / totalShares;
if (notional < minUsdcOut) revert SlippageExceeded(); // one global floor
```

IMPACT

Multi-leg buys and rebalance steps revert whenever any one leg slips below its individual floor, with revert probability compounding as the portfolio adds legs. No funds are lost, but execution reliability degrades for exactly the fast-moving assets the protocol is built around, and a buyer cannot let an outperforming leg offset an underperforming one.

RECOMMENDATION

Replace the per-leg floors on the buy and rebalance paths with a single aggregate slippage bound on the net LT value acquired, mirroring `minUsdcOut` in `withdrawLtsTo`. One global floor lets legs net against each other and reverts only when the aggregate execution is genuinely worse than the user accepts.

L-08 LOW ACKNOWLEDGED

`sell()` payout can leave small sellers with LT dust

`withdrawLtsTo` pays `min(notional, idle)` in USDC first, then covers the deficit with raw LT tokens, a proportional slice of every leg:

```
uint256 usdcOut = notional ⩽ idle ? notional : idle;
if (usdcOut > 0) USDC.safeTransfer(recipient, usdcOut);
uint256 remaining = notional - usdcOut;
if (remaining > 0) {
    for (uint256 i = 0; i < n; i++) {
        ...
        uint256 ltOut = (bal * remaining) / totalLtValue;
        if (ltOut == 0) continue;
        lt.safeTransfer(recipient, ltOut);
    }
}
```

NAV values LTs at gross `ltToBaseAmount(bal)` with no fee deduction, so `notional` treats USDC and LT face value as interchangeable when their realizable values are not. An LT recipient must later call `lt.redeem` or `lt.prepareRedeem` on Bounce, paying `redemptionFee * targetLeverage` plus the executor fee on the async path. This creates two problems:

1. Two sellers burning the same share count get the same face value but different realizable USDC. Whoever drains the idle USDC walks away clean, and the next seller eats the redemption fee on their LT slice.
2. Idle USDC is normally near zero after deploys, so the LT path is the common case. For a small sell against a many-leg portfolio, each slice can be worth only a few dollars, below Bounce's `minTransactionSize`, and both `redeem` and `prepareRedeem` revert with `BelowMinTransactionSize` under that floor, so the seller cannot convert the dust LTs to USDC at all.

RECOMMENDATION

Pay sellers in USDC rather than raw LTs. Either redeem the required LTs to USDC inside `withdrawLtsTo` and distribute net USDC, or schedule the withdrawal like Bounce so the seller receives USDC once it settles.

L-09 LOW ACKNOWLEDGED

`buy()` lets the caller waive mint slippage on a deploy of the entire shared idle pool

When calling `buy()` users supply `minLtOuts` to `deployUsdc`, which hands it to `_deployIdle` and then `_mintLTs`, where it becomes the slippage check on Bounce LT mints:

```
USDC.forceApprove(address(lt), usdcToAllocate);
uint256 ltOut = lt.mint(address(this), usdcToAllocate, minLtOuts[i]);
if (ltOut < minLtOuts[i]) revert SlippageExceeded();
```

The contract applies no floor or sanity check on these values, so any caller can pass zeros and waive the treasury's mint slippage protection entirely. The important part is that `_deployIdle` deploys the treasury's full idle balance, not just the caller's `usdcIn`:

```
uint256 idle = USDC.balanceOf(address(this));
uint256 threshold = minDeployUsdc();
if (idle < threshold) {
    emit DeployDeferred(idle, threshold);
    return;
}
_mintLTs(idle, minLtOuts);
```

RECOMMENDATION

Apply a protocol-fixed slippage percentage, or enforce slippage in `executeRebalanceStep` instead of deposits.

L-10 LOW FIXED

Rebalance is limited by Bounce's minimum redeem transaction size

`executeRebalanceStep` computes `shrinkLt` per leg and forwards it to either `lt.redeem` or `lt.prepareRedeem`. Bounce's `LeveragedToken` enforces `baseAmount_ ⩾ globalStorage.minTransactionSize()` on both paths, but the treasury never compares `expectedBaseOut` against that threshold:

```
// AgentTreasury.sol — executeRebalanceStep sell loop
uint256 expectedBaseOut = lt.ltToBaseAmount(shrinkLt);
if (expectedBaseOut < minRedeemUsdc[i]) revert SlippageExceeded();
// no check against lt.globalStorage.minTransactionSize()
int256 bufferRaw = LT_HELPER.getLeveragedTokenBufferAssetValue(address(lt));
uint256 buffer = bufferRaw < 0 ? 0 : uint256(bufferRaw);
if (expectedBaseOut ⩽ buffer) {
    uint256 actualBaseOut = lt.redeem(address(this), shrinkLt, minRedeemUsdc[i]); // reverts
BelowMinTransactionSize
    ...
} else {
    ...
    lt.prepareRedeem(shrinkLt); // reverts BelowMinTransactionSize
    ...
}
```

When `expectedBaseOut < minTransactionSize`, the entire `executeRebalanceStep` reverts, so one undersized leg blocks all the others in the same call. The rebalancer can only sidestep it by setting `maxRedeemLt[i] = 0` and `minRedeemUsdc[i] = 0` for that leg, which then leaves the leg permanently above target.

IMPACT

Rebalance is bricked any time a leg's required shrink falls below Bounce's `minTransactionSize`. The rebalancer must manually pre-compute every leg's shrink off-chain and zero out the offending caps to make the call go through.

RECOMMENDATION

Skip an undersized leg instead of reverting the whole step, or document the behaviour so the rebalancer chooses `maxRedeemLt` values that clear Bounce's minimum.

L-11 LOW FIXED

`buy` can accept payment yet mint zero shares

`buy` pulls the user's USDC, deploys it, and mints `_quoteBuy`'s output without checking it is non-zero:

```
agentOut = _quoteBuy(usdcIn, supplyBefore, navBefore);
if (agentOut < minAgentOut) revert SlippageExceeded();   // only the user-supplied bound
TREASURY.deployUsdc(usdcIn, minLtOuts);                  // payment taken
_mint(recipient, agentOut);                              // may mint 0
```

A non-zero payment can mint 0 AGENT, value in and nothing out. Normally the `1e12` scaling keeps the quote above 1, but `sell` checks `supplyBefore == 0` before the burn, so a holder can drive supply to 1, where `premium(1) == 1e18` collapses the quote to `floor(usdcIn / nav)`. Since `nav()` also counts donations, a sole holder can then make a `minAgentOut == 0` buyer mint 0 and pocket their deposit. `SupplyCollapseInflationPoC.t.sol` confirms both steps on the HyperEVM fork with no mocks.

IMPACT

A real invariant defect, but heavily gated: any non-zero `minAgentOut` reverts, it needs a sole-holder genesis state, and the loss is the buyer's own deposit, hence Low.

RECOMMENDATION

`require(agentOut > 0)` in `buy`. A paid buy must never mint 0.

L-12 LOW FIXED

Paused-leg skip in `_mintLTs` silently ignores the buyer's per-leg `minLtOuts` floor

`_mintLTs` has two branches that produce zero LT for a leg, and it handles them inconsistently. The `usdcToAllocate == 0` branch honors the caller's per-leg floor, while the `lt.mintPaused()` branch silently skips the leg without reading it:

```
if (usdcToAllocate == 0) {
    if (minLtOuts[i] != 0) revert SlippageExceeded();   // floor honored
    continue;
}
if (lt.mintPaused()) {
    emit MintSkippedPaused(sym, usdcToAllocate);
    continue;                                            // floor IGNORED
}
USDC.forceApprove(address(lt), usdcToAllocate);
uint256 ltOut = lt.mint(address(this), usdcToAllocate, minLtOuts[i]);
if (ltOut < minLtOuts[i]) revert SlippageExceeded();
```

A buyer who passes `minLtOuts[i] != 0` is explicitly demanding at least that many units of leg i or a revert. If that leg's LT is paused, the buy nonetheless succeeds with zero units minted and the leg's USDC parked idle. That the sibling branch does revert on the same `minLtOuts[i]` shows the floor is meant to be enforced.

1. A treasury runs 50/50 `HYPE5L` / `BTC5L`, both legs deployed.
2. `HYPE5L` mint is paused, its `bps` still 5000.
3. A buyer calls `buy(..., minLtOuts = [1, 0])`, demanding at least 1 unit of `HYPE5L`.
4. The buy succeeds with zero `HYPE5L` minted, that leg's USDC sits idle, and `minLtOuts[0] = 1` is silently ignored.

IMPACT

The buyer's per-leg exposure floor is unenforceable on any paused leg: they receive unlevered idle-USDC backing instead of the leveraged exposure they required, with no transaction-level signal to abort.

RECOMMENDATION

Mirror the `usdcToAllocate == 0` branch so the floor is honored before skipping a paused leg. Callers who pass `minLtOuts[i] == 0` keep the intended skip-paused resilience.

```
  if (lt.mintPaused()) {
+     if (minLtOuts[i] != 0) revert SlippageExceeded();
      emit MintSkippedPaused(sym, usdcToAllocate);
      continue;
  }
```

L-13 LOW FIXED

Fee is pushed to a hardcoded, unchangeable recipient

`AgentCurve.buy()` and `AgentTreasury.withdrawLtsTo()` send the 1% fee to a hardcoded `FEE_RECIPIENT` on every trade, with no setter:

```
address private constant FEE_RECIPIENT = 0xb2feD3aCf6e30e0f1902A2b190C88C9a0a68eDC3;
...
if (fee > 0) USDC.safeTransfer(FEE_RECIPIENT, fee);
```

IMPACT

If `FEE_RECIPIENT` is USDC-blacklisted, the transfer reverts and every buy and sell reverts with it. Being a `constant` with no setter, it cannot be redirected, so only a full redeploy fixes it.

RECOMMENDATION

Make the recipient and bps configurable via a setter, or accrue fees for pull-withdrawal so a failing transfer cannot block trading.